

## Punto 2.8 - Claves primarias, alternativas y compuestas

**Ejercicio guiado:** consolidar las reglas de identidad del modelo `AceriaData` distinguiendo qué papel cumple una clave primaria, cuándo una clave alternativa representa un identificador de negocio y por qué una clave compuesta identifica una relación mediante la combinación de varias propiedades.

**Contexto del proyecto:** 2.8 parte íntegramente del estado 2.7. No se rehacen las relaciones ni la configuración Fluent ya existente. El cambio de esquema de este punto consiste en añadir tres claves alternativas reales: `NumeroOrden` en `OrdenFabricacion`, `Codigo` en `Aleacion` y `NumeroCertificado` en `CertificadoCalidad`. La clave compuesta de `OrdenAleacion` ya existía y se conserva. Los índices adicionales y las restricciones CHECK se reservan para 2.9.

### Objetivos de aprendizaje

- Distinguir clave primaria, clave alternativa y clave compuesta.
- Reconocer una clave alternativa como clave candidata del modelo, no como simple índice de rendimiento.
- Configurar `HasAlternateKey` y asignar un nombre estable a la restricción.
- Mantener la PK simple `Id` de las entidades principales.
- Mantener la PK compuesta `OrdenFabricacionId + AleacionId` de `OrdenAleacion`.
- Añadir las claves alternativas reales de `OrdenFabricacion`, `Aleacion` y `CertificadoCalidad`.
- Interpretar la migración real `M2_2_8` y sus tres `AddUniqueConstraint`.
- Inspeccionar claves con el metadata de EF Core mediante `GetKeys()` y `FindPrimaryKey()`.
- Verificar las restricciones en SQL Server LocalDB.
- Comprobar que una clave alternativa rechaza un identificador de negocio duplicado.
- Diferenciar una alternate key de los índices que se estudiarán en 2.9.

### Paso 1: Abrir el estado autónomo de 2.8

```
cd M02/PROYECTO/2.8
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El checkpoint contiene el estado completo y ejecutable del punto. Si reproduces la evolución manualmente, parte de 2.7 y aplica únicamente los cambios descritos aquí.

### Paso 2: Identificar las tres formas de identidad presentes

En `AceriaData` conviven tres situaciones diferentes:

- **PK simple:** `OrdenFabricacion.Id`, `Aleacion.Id` o `CertificadoCalidad.Id` identifican técnicamente una fila.
- **Clave alternativa:** `NumeroOrden`, `Codigo` y `NumeroCertificado` son identificadores de negocio que también deben ser únicos.
- **PK compuesta:** `OrdenAleacion` se identifica por la combinación `OrdenFabricacionId + AleacionId`.

La diferencia es importante: una alternate key forma parte del modelo de identidad de EF Core y puede ser objetivo de relaciones. No debe añadirse sólo porque se quiera acelerar una búsqueda; para eso existen los índices.

### Paso 3: Mantener la PK y añadir la alternate key de `OrdenFabricacion`

El bloque relevante del modelo efectivo es:

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.ToTable("OrdenesFabricacion");
    entity.HasKey(o => o.Id);

    entity.HasAlternateKey(o => o.NumeroOrden)
        .HasName("AK_OrdenesFabricacion_NumeroOrden");
});
```

`HasKey(o => o.Id)` mantiene `Id` como clave primaria. `HasAlternateKey(o => o.NumeroOrden)` declara `NumeroOrden` como segunda clave candidata y `HasName` fija el nombre de la restricción que aparecerá en SQL Server.

No se añade además un `HasIndex(...).IsUnique()` sobre `NumeroOrden` en este punto: sería mezclar una regla de identidad con el contenido específico de índices de 2.9.

## Paso 4: Mantener la PK de PlanchaAcero

`PlanchaAcero` continúa usando una PK simple:

```
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.ToTable("PlanchasAcero");
    entity.HasKey(x => x.Id);
});
```

No hay una nueva alternate key para esta entidad en 2.8. El resto de su configuración —precisión, default de `Activa` y relación con `OrdenFabricacion`— se hereda sin cambios desde 2.7.

## Paso 5: Añadir Codigo como clave alternativa de Aleacion

```
modelBuilder.Entity<Aleacion>(entity =>
{
    entity.ToTable("Aleaciones");
    entity.HasKey(a => a.Id);

    entity.HasAlternateKey(a => a.Codigo)
        .HasName("AK_Aleaciones_Codigo");
});
```

`Id` sigue siendo la PK técnica. `Codigo` pasa a ser un identificador de negocio único. La migración real de este punto materializa esa decisión como una restricción UNIQUE llamada `AK_Aleaciones_Codigo`.

No se añade una segunda clave alternativa compuesta `Nombre + Codigo`: no forma parte del estado final 2.8 y cambiaría el modelo que debe recibir 2.9.

## Paso 6: Revisar EstadoOrden y no inventar una clave nueva

`EstadoOrden` mantiene:

```
modelBuilder.Entity<EstadoOrden>(entity =>
{
    entity.ToTable("EstadosOrden");
    entity.HasKey(e => e.Id);
});
```

`Nombre` continúa siendo una propiedad requerida con longitud máxima, pero **no se convierte en alternate key en este punto**. Esto permite que la práctica coincida exactamente con el checkpoint y con la migración `M2_2_8`.

## Paso 7: Mantener la PK de DetalleOrden y su relación 1:1

```
modelBuilder.Entity<DetalleOrden>(entity =>
{
    entity.ToTable("DetallesOrden");
    entity.HasKey(d => d.Id);

    entity.HasOne(d => d.Orden)
        .WithOne(o => o.Detalle)
        .HasForeignKey<DetalleOrden>(d => d.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
```

La identidad principal sigue en `Id`. La unicidad necesaria para la relación 1:1 se deriva de la propia relación; no se crea una alternate key artificial sobre `OrdenId`.

## Paso 8: Añadir NumeroCertificado como clave alternativa

```
modelBuilder.Entity<CertificadoCalidad>(entity =>
{
    entity.ToTable("CertificadosCalidad");
    entity.HasKey(c => c.Id);

    entity.HasAlternateKey(c => c.NumeroCertificado)
        .HasName("AK_CertificadosCalidad_NumeroCertificado");
});
```

Un certificado conserva su `Id` técnico y, al mismo tiempo, `NumeroCertificado` se convierte en identificador de negocio único.

## Paso 9: Mantener la clave primaria compuesta de OrdenAleacion

```
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");

    entity.HasKey(x => new
    {
        x.OrdenFabricacionId,
        x.AleacionId
    });
});
```

La combinación de ambas FK identifica una sola asignación de una aleación a una orden. No se cambia el orden de las columnas ni se introduce una segunda PK: se conserva la definición ya existente y coherente con los estados anteriores.

## Paso 10: Compilar antes de generar la migración

```
dotnet build AceriaData.sln --configuration Release
```

El build debe ser correcto antes de pedir a EF Core que compare el modelo 2.8 con el snapshot de 2.7.

## Paso 11: Generar la migración M2\_2\_8 al reproducir la evolución

Si partes manualmente del estado 2.7:

```
dotnet ef migrations add M2_2_8
```

El checkpoint final ya contiene:

```
Migrations/20260927204815_M2_2_8.cs
```

No generes de nuevo la misma migración dentro de `M02/PROYECTO/2.8`.

## Paso 12: Leer la migración real, no una migración hipotética

`M2_2_8` añade exactamente tres restricciones:

```
migrationBuilder.AddUniqueConstraint(
    name: "AK_OrdenesFabricacion_NumeroOrden",
    table: "OrdenesFabricacion",
    column: "NumeroOrden");

migrationBuilder.AddUniqueConstraint(
    name: "AK_CertificadosCalidad_NumeroCertificado",
    table: "CertificadosCalidad",
    column: "NumeroCertificado");

migrationBuilder.AddUniqueConstraint(
    name: "AK_Aleaciones_Codigo",
    table: "Aleaciones",
    column: "Codigo");
```

No elimina ni recrea las PK existentes y no añade una alternate key a `EstadoOrden.Nombre`. Esa diferencia permite separar el objetivo pedagógico de 2.8 de configuraciones que no pertenecen al modelo final.

## Paso 13: Aplicar el historial y verificar SQL Server

```
dotnet ef database update
dotnet ef migrations list
```

La lista debe contener `M2_2_8` después de `M2_2_7`.

En Visual Studio puedes abrir **Ver → Explorador de objetos de SQL Server**, expandir `(localdb)\MSSQLLocalDB` → **Bases de datos** → **AceriaDB** y revisar las restricciones de las tablas implicadas.

Comprueba especialmente:

- `AK_OrdenesFabricacion_NumeroOrden`;
- `AK_Aleaciones_Codigo`;
- `AK_CertificadosCalidad_NumeroCertificado`;
- la PK compuesta de `OrdenesAleaciones`.

## Paso 14: Ejecutar e inspeccionar las claves del modelo

```
dotnet run --project AceriaData.Console.csproj --configuration Release
```

El checkpoint obtiene los metadatos reales:

```
var entity = context.Model.FindEntityType(typeof(OrdenFabricacion))!;

Console.WriteLine(
    $"2.8 OK | Alternate keys: {entity.GetKeys().Count()} | " +
    $"PK: {string.Join(", ", entity.FindPrimaryKey().Properties.Select(x => x.Name))}");
```

`GetKeys()` devuelve las claves configuradas para la entidad, incluida la primaria y las alternativas. `FindPrimaryKey()` permite comprobar qué propiedades forman la PK efectiva.

## Paso 15: Resolver el reto de NumeroOrden duplicado

**Reto:** demostrar que la alternate key de `NumeroOrden` protege la identidad de negocio y no es sólo metadata.

El checkpoint contiene el bloque comentado `RETO 2.8 - NUMEROORDEN DUPLICADO`. Al activarlo se guarda primero una orden con:

```
NumeroOrden = "OF-M2-RETO-28"
```

y, desde un segundo ámbito y otro `DbContext`, se intenta guardar otra orden con el mismo valor.

La segunda operación se envuelve en:

```
try
{
    retoContext.SaveChanges();
    Console.WriteLine(
        "Reto 2.8 ERROR: se permitió NumeroOrden duplicado.");
}
catch (DbUpdateException)
{
    Console.WriteLine(
        "Reto 2.8: NumeroOrden duplicado rechazado");
}
```

Resultado esperado:

```
Reto 2.8: NumeroOrden duplicado rechazado
```

Se usan dos operaciones y un segundo contexto para que la demostración llegue realmente a SQL Server. El esquema se prepara mediante `Migrations`; no se sustituye por `EnsureCreated()`.

## Paso 16: Razonar sobre la PK compuesta de OrdenAleacion

La PK de `OrdenAleacion` impide que exista dos veces la misma pareja `OrdenFabricacionId + AleacionId`. La regla se puede inspeccionar sin cambiar el modelo final:

```
var ordenAleacionEntity =
    context.Model.FindEntityType(typeof(OrdenAleacion));

var pk = ordenAleacionEntity.FindPrimaryKey();

Console.WriteLine(string.Join(
    ", ",
    pk.Properties.Select(p => p.Name)));
```

El resultado debe identificar las dos propiedades de la clave. Si se intenta persistir otra fila con la misma combinación, SQL Server protege la unicidad de la PK compuesta.

## Paso 17: Delimitar el alcance antes de pasar a 2.9

En 2.8 **no** se añaden:

- índices únicos o compuestos adicionales con `HasIndex`;
- restricciones CHECK;
- una alternate key en `EstadoOrden.Nombre`;
- una alternate key compuesta extra en `Aleacion`;
- cambios de `DeleteBehavior`;
- nuevas entidades;
- `EnsureCreated()` como sustituto del historial de `Migrations`.

El objetivo es cerrar correctamente las reglas de identidad antes de estudiar índices y restricciones en el punto siguiente.

## Errores comunes

| Error                                                                                                  | Causa                                                              | Corrección                                                                                        |
|--------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Confundir alternate key con índice único                                                               | Ambas pueden imponer unicidad, pero expresan intenciones distintas | Usar <code>HasAlternateKey</code> para una clave candidata; estudiar <code>HasIndex</code> en 2.9 |
| Duplicar <code>HasAlternateKey</code> y <code>HasIndex(...).IsUnique()</code> sobre la misma propiedad | Se mezclan dos configuraciones de unicidad                         | Mantener sólo la regla que corresponde al objetivo del punto                                      |
| Omitir una propiedad de la PK compuesta                                                                | La identidad de <code>OrdenAleacion</code> queda incompleta        | Mantener <code>OrdenFabricacionId + AleacionId</code>                                             |
| Añadir una alternate key que no aparece en la migración real                                           | El MD deja de representar el checkpoint                            | Contrastar siempre modelo, snapshot y <code>M2_2_8</code>                                         |
| Aplicar una UNIQUE con datos duplicados existentes                                                     | SQL Server no puede crear la restricción                           | Limpiar o corregir los datos antes de aplicar la migración                                        |
| Probar duplicados en el mismo <code>ChangeTracker</code> y confundir el error                          | EF Core puede detectar conflictos antes de llegar a SQL Server     | Usar operaciones separadas cuando se quiera demostrar la restricción de base de datos             |
| Usar <code>EnsureCreated()</code>                                                                      | Se evita el historial acumulativo de Migrations                    | Mantener <code>Database.Migrate()</code> / <code>dotnet ef database update</code>                 |

## Analogía final

En una acería, la PK es el identificador técnico grabado en el registro interno de una pieza. Una alternate key es otro identificador reconocido por el negocio —por ejemplo, el número oficial de una orden o certificado— que también debe ser irrepetible. La PK compuesta de `OrdenAleacion` se parece a una ficha cuya identidad depende simultáneamente de la orden y de la aleación: repetir exactamente la misma pareja significaría duplicar la misma asignación.

## Resultado esperado

Al finalizar 2.8, `AceriaData` conserva todas las reglas válidas de 2.7, mantiene las PK simples y la PK compuesta de `OrdenAleacion`, y añade tres claves alternativas reales: `NumeroOrden`, `Codigo` y `NumeroCertificado`. La migración `M2_2_8` materializa esas tres restricciones UNIQUE. La ejecución permite inspeccionar las claves del modelo y el reto confirma que SQL Server rechaza un `NumeroOrden` duplicado.

## Conexión con el siguiente punto

2.9 partirá de este estado para trabajar específicamente con índices y restricciones, sin confundir una clave candidata con una estructura creada únicamente para búsqueda, ordenación o integridad adicional.

## Código acumulativo completo del estado 2.8

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }

```

```

[Required]
[MaxLength(50)]
public string Nombre { get; set; } = string.Empty;
[MaxLength(250)]
public string Descripcion { get; set; } = string.Empty;
public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>

```



```

{
    entity.ToTable("OrdenesFabricacion");
    entity.HasKey(o => o.Id);
    entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
    entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
    entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
    entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
    entity.Property(o => o.Observaciones).HasMaxLength(500);
    entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
});
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.ToTable("PlanchasAcero");
    entity.HasKey(x => x.Id);
    entity.Property(x => x.Peso).HasPrecision(18, 3);
    entity.Property(x => x.Activa).HasDefaultValue(true);
    entity.HasOne(x => x.Orden)
        .WithMany(o => o.Planchas)
        .HasForeignKey(x => x.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
modelBuilder.Entity<Aleacion>(entity =>
{
    entity.ToTable("Aleaciones");
    entity.HasKey(a => a.Id);
    entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
    entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
    entity.Property(a => a.Descripcion).HasMaxLength(500);
    entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");
});
modelBuilder.Entity<EstadoOrden>(entity =>
{
    entity.ToTable("EstadosOrden");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
    entity.Property(e => e.Descripcion).HasMaxLength(250);
    entity.Property(e => e.Activo).HasDefaultValue(true);
});
modelBuilder.Entity<DetalleOrden>(entity =>
{
    entity.ToTable("DetallesOrden");
    entity.HasKey(d => d.Id);
    entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
    entity.Property(d => d.Notas).HasMaxLength(500);
    entity.HasOne(d => d.Orden)
        .WithOne(o => o.Detalle)
        .HasForeignKey<DetalleOrden>(d => d.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});

modelBuilder.Entity<CertificadoCalidad>(entity =>
{
    entity.ToTable("CertificadosCalidad");
    entity.HasKey(c => c.Id);
    entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
    entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
    entity.HasAlternateKey(c =>
c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");
    entity.HasOne(c => c.Orden)
        .WithOne(o => o.Certificado)
        .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");
    entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
    entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
    entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
    entity.Property(x =>

```

```

x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
    entity.HasOne(x => x.Orden)
        .WithMany(o => o.OrdenesAleaciones)
        .HasForeignKey(x => x.OrdenFabricacionId)
        .OnDelete(DeleteBehavior.Cascade);
    entity.HasOne(x => x.Aleacion)
        .WithMany(a => a.OrdenesAleaciones)
        .HasForeignKey(x => x.AleacionId)
        .OnDelete(DeleteBehavior.Restrict);
    });
}
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o =>
o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")

```

```

        ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

var services = new ServiceCollection();
services.AddDbContext<AceriaDbContext>(options =>
    options
        .UseSqlServer(connectionString, sql =>
            {
                sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                sql.CommandTimeout(60);
            })
        .LogTo(global::System.Console.WriteLine, new[] {
            "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
        .EnableDetailedErrors());

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
});

```

```

        EstadoRelacion = "Activa"
    });
    context.OrdenesFabricacion.Add(orden);
    context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
Activo = true });
    context.SaveChanges();

    var entity = context.Model.FindEntityType(typeof(OrdenFabricacion));
    global::System.Console.WriteLine($"2.8 OK | Alternate keys: {entity.GetKeys().Count()} | PK:
{string.Join(", ", entity.FindPrimaryKey().Properties.Select(x => x.Name))}");
    }
}

```

## Explicación línea a línea del código acumulativo 2.8

Las referencias siguientes usan la numeración real de M02/PROYECTO/2.8/Program.cs. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `[Table("OrdenesFabricacion")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 11: `public class OrdenFabricacion` → declara la clase OrdenFabricacion que forma parte del estado acumulativo del ejercicio.

Línea 12: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 13: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 14: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 15: `[Required]` → marca la propiedad siguiente como requerida.

Línea 16: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 17: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad NumeroOrden de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 18: `[Required]` → marca la propiedad siguiente como requerida.

Línea 19: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 20: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad Cliente de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 21: `public DateTime FechaCreacion { get; set; }` → declara la propiedad FechaCreacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 22: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad FechaEntrega de tipo DateTime? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 23: `[Required]` → marca la propiedad siguiente como requerida.

Línea 24: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 25: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad Estado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 26: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 27: `public string? Observaciones { get; set; }` → declara la propiedad Observaciones de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 28: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad Planchas de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 29: `public DetalleOrden? Detalle { get; set; }` → declara la propiedad Detalle de tipo DetalleOrden? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 30: `public CertificadoCalidad? Certificado { get; set; }` → declara la propiedad Certificado de tipo CertificadoCalidad? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 31: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 32: `}` → cierra el bloque de código actual.

Línea 34: `[Table("PlanchasAcero")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 35: `public class PlanchaAcero` → declara la clase PlanchaAcero que forma parte del estado acumulativo del ejercicio.

Línea 36: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 37: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 38: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 39: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 40: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 41: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 42: `public double Largo { get; set; }` → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 43: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 44: `public decimal Peso { get; set; }` → declara la propiedad Peso de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.

Línea 45: `public bool Activa { get; set; } = true;` → declara la propiedad Activa de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 46: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 47: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 48: `}` → cierra el bloque de código actual.

Línea 50: `[Table("Aleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 51: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.

Línea 52: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 53: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 54: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 55: `[Required]` → marca la propiedad siguiente como requerida.

Línea 56: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 57: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 58: `[Required]` → marca la propiedad siguiente como requerida.

Línea 59: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 60: `public string Codigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 61: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 62: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 63: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 64: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 65: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 66: `}` → cierra el bloque de código actual.

Línea 68: `[Table("EstadosOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 69: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.

Línea 70: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 71: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 72: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 73: `[Required]` → marca la propiedad siguiente como requerida.

Línea 74: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 75: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 76: `[MaxLength(250)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 77: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 78: `public bool Activo { get; set; } = true;` → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 79: `}` → cierra el bloque de código actual.

Línea 81: `[Table("DetallesOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 82: `public class DetalleOrden` → declara la clase DetalleOrden que forma parte del estado acumulativo del ejercicio.

Línea 83: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 84: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 85: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 86: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 87: `[Required]` → marca la propiedad siguiente como requerida.

Línea 88: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 89: `public string ComposicionQuimica { get; set; } = string.Empty;` → declara la propiedad ComposicionQuimica de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 90: `public double TemperaturaColada { get; set; }` → declara la propiedad TemperaturaColada de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 91: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 92: `public string? Notas { get; set; }` → declara la propiedad Notas de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 93: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 94: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 95: `}` → cierra el bloque de código actual.

Línea 97: `[Table("CertificadosCalidad")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 98: `public class CertificadoCalidad` → declara la clase CertificadoCalidad que forma parte del estado acumulativo del ejercicio.

Línea 99: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 100: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 101: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 102: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 103: `[Required]` → marca la propiedad siguiente como requerida.

Línea 104: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 105: `public string NumeroCertificado { get; set; } = string.Empty;` → declara la propiedad NumeroCertificado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.



Línea 106: `public DateTime FechaEmision { get; set; }` → declara la propiedad FechaEmision de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 107: `[Required]` → marca la propiedad siguiente como requerida.

Línea 108: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 109: `public string OrganismoCertificador { get; set; } = string.Empty;` → declara la propiedad OrganismoCertificador de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 110: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 111: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 112: `}` → cierra el bloque de código actual.

Línea 114: `[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]` → declara una clave primaria compuesta con las propiedades indicadas.

Línea 115: `[Table("OrdenesAleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 116: `public class OrdenAleacion` → declara la clase OrdenAleacion que forma parte del estado acumulativo del ejercicio.

Línea 117: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 118: `public int OrdenFabricacionId { get; set; }` → declara la propiedad OrdenFabricacionId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 119: `public int AleacionId { get; set; }` → declara la propiedad AleacionId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 120: `public DateTime FechaAsignacion { get; set; } = DateTime.Now;` → declara la propiedad FechaAsignacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 121: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 122: `public decimal CantidadUtilizada { get; set; }` → declara la propiedad CantidadUtilizada de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.

Línea 123: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 124: `public string EstadoRelacion { get; set; } = "Activa";` → declara la propiedad EstadoRelacion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 125: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 126: `public Aleacion Aleacion { get; set; } = null!;` → declara la propiedad Aleacion de tipo Aleacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 127: `}` → cierra el bloque de código actual.

Línea 129: `public class AceriaDbContext : DbContext` → declara la clase AceriaDbContext que forma parte del estado acumulativo del ejercicio.

Línea 130: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 131:

`public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → declara el constructor o método AceriaDbContext con los parámetros necesarios para este componente.

Línea 133: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.



Línea 134: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 135: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 136: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 137: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 138: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 139: `public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 142: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 143: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 144: `base.OnModelCreating(modelBuilder);` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 145: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 146: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 147: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 148: `entity.HasKey(o => o.Id);` → define la clave primaria de la entidad.

Línea 149: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 150: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 151: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 152: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 153: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 154: `entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");` → define una clave alternativa que también debe ser única.

Línea 155: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 156: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 157: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 158: `entity.ToTable("PlanchasAcero");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 159: `entity.HasKey(x => x.Id);` → define la clave primaria de la entidad.

Línea 160: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 161: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 162: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 163: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 164: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 165: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 166: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 167: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 168: `modelBuilder.Entity<Aleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 169: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 170: `entity.ToTable("Aleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 171: `entity.HasKey(a => a.Id);` → define la clave primaria de la entidad.

Línea 172: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 173: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);` → selecciona una propiedad escalar para continuar su configuración.

Línea 174: `entity.Property(a => a.Descripcion).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 175: `entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");` → define una clave alternativa que también debe ser única.

Línea 176: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 177: `modelBuilder.Entity<EstadoOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 178: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 179: `entity.ToTable("EstadosOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 180: `entity.HasKey(e => e.Id);` → define la clave primaria de la entidad.

Línea 181: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 182: `entity.Property(e => e.Descripcion).HasMaxLength(250);` → selecciona una propiedad escalar para continuar su configuración.

Línea 183: `entity.Property(e => e.Activo).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 184: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 185: `modelBuilder.Entity<DetalleOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 186: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 187: `entity.ToTable("DetallesOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 188: `entity.HasKey(d => d.Id);` → define la clave primaria de la entidad.

Línea 189: `entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 190: `entity.Property(d => d.Notas).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 191: `entity.HasOne(d => d.Orden)` → inicia la configuración de una navegación de referencia.

Línea 192: `.WithOne(o => o.Detalle)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 193: `.HasForeignKey<DetalleOrden>(d => d.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 194: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 195: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 196: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 198: `modelBuilder.Entity<CertificadoCalidad>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 199: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 200: `entity.ToTable("CertificadosCalidad");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 201: `entity.HasKey(c => c.Id);` → define la clave primaria de la entidad.

Línea 202: `entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 203: `entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 204: `entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");` → define una clave alternativa que también debe ser única.

Línea 205: `entity.HasOne(c => c.Orden)` → inicia la configuración de una navegación de referencia.

Línea 206: `.WithOne(o => o.Certificado)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 207: `.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 208: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 209: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 210: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 211: `modelBuilder.Entity<OrdenAleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 212: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 213: `entity.ToTable("OrdenesAleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 214: `entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });` → define la clave primaria de la entidad.

Línea 215: `entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 216: `entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 217: `entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");` → selecciona una propiedad escalar para continuar su configuración.

Línea 218: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 219: `.WithMany(o => o.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 220: `.HasForeignKey(x => x.OrdenFabricacionId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 221: `.OnDelete(DeleteBehavior.Cascade);` → define el comportamiento de eliminación de la relación.

Línea 222: `entity.HasOne(x => x.Aleacion)` → inicia la configuración de una navegación de referencia.

Línea 223: `.WithMany(a => a.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 224: `.HasForeignKey(x => x.AleacionId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 225: `.OnDelete(DeleteBehavior.Restrict);` → define el comportamiento de eliminación de la relación.

Línea 226: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 227: `}` → cierra el bloque de código actual.

Línea 228: `}` → cierra el bloque de código actual.

Línea 230: `public interface IOrdenRepositorio` → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.

Línea 231: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 232: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 233: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 234: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.

Línea 235: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 236: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 237: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 238: `}` → cierra el bloque de código actual.

Línea 240: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase `OrdenRepositorio` que forma parte del estado acumulativo del ejercicio.

Línea 241: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 242: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 243: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → declara el constructor o método `OrdenRepositorio` con los parámetros necesarios para este componente.

Línea 245: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado de la consulta antes de materializarlo.

Línea 246: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante `Find`, aprovechando primero el `ChangeTracker` y después la base de datos.

Línea 247: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o `null` cuando no existe.

Línea 248: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 249: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca la entidad indicada para borrado al confirmar los cambios.

Línea 250: `public int Guardar() => _context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 251: `}` → cierra el bloque de código actual.

Línea 253: `public interface IServicioOrdenes` → declara el contrato `IServicioOrdenes` para desacoplar consumidores e implementación.

Línea 254: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 255: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 256: `}` → cierra el bloque de código actual.

Línea 258: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase `ServicioOrdenes` que forma parte del estado acumulativo del ejercicio.

Línea 259: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 260: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado `_repositorio` de tipo `IOrdenRepositorio` que conserva una dependencia de la clase.

Línea 261: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 262: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor o método `ServicioOrdenes` con los parámetros necesarios para este componente.

Línea 263: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 264: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 265: `_context = context;` → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.

Línea 266: `}` → cierra el bloque de código actual.

Línea 268: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 269: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 270: `var orden = _repositorio.ObtenerPorId(ordenId);` → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.

Línea 271: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.

Línea 272: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta construida.

Línea 273: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado calculado al código que invocó el método.

Línea 274: `}` → cierra el bloque de código actual.

Línea 275: `}` → cierra el bloque de código actual.

Línea 277: `public static class Program` → declara la clase Program que forma parte del estado acumulativo del ejercicio.

Línea 278: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 279: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 280: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 281: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 282: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base usada para localizar los archivos de configuración.

Línea 283: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora appsettings.json a la configuración.

Línea 284: `.AddEnvironmentVariables()` → incorpora variables de entorno como fuente adicional de configuración.

Línea 285: `.Build();` → materializa la configuración o el objeto encadenado construido en las líneas anteriores.

Línea 287: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena de conexión AceriaDB desde la configuración.

Línea 288: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.

Línea 290: `var services = new ServiceCollection();` → crea la colección donde se registran las dependencias de la aplicación.

Línea 291: `services.AddDbContext<AceriaDbContext>(options =>` → registra el DbContext en el contenedor de inyección de dependencias.

Línea 292: `options` → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.



Línea 293: `.UseSqlServer(connectionString, sql =>` → selecciona SQL Server como proveedor de EF Core.

Línea 294: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 295: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios del proveedor SQL Server.

Línea 296: `sql.CommandTimeout(60);` → establece el tiempo máximo permitido para los comandos SQL.

Línea 297: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 298: `.LogTo(global::System.Console.WriteLine, new[]  
{ "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura la salida del logging de EF Core.

Línea 299: `.EnableDetailedErrors();` → activa errores detallados para facilitar el diagnóstico del laboratorio.

Línea 301: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 302: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 304: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.

Línea 305: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 306: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 307: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.

Línea 308: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 310: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 311: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio desde el contenedor de dependencias.

Línea 312: `context.Database.EnsureDeleted();` → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.

Línea 313: `context.Database.Migrate();` → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.

Línea 315: `var orden = new OrdenFabricacion` → crea la entidad OrdenFabricacion que concentra el grafo de datos del escenario acumulativo.

Línea 316: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 317: `NumeroOrden = "OF-M2-0001",` → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 318: `Cliente = "Constructora del Norte",` → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 319: `FechaCreacion = DateTime.UtcNow,` → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.



Línea 320: `FechaEntrega = DateTime.Today.AddDays(14)`, → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 321: `Estado = "Pendiente"`, → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.

Línea 322: `Observaciones = "Orden de validación M2"`, → añade texto opcional para comprobar el mapeo de Observaciones.

Línea 323: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 324: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 325: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.

Línea 326: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 327: `Espesor = 10.5`, → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 328: `Ancho = 1500`, → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 329: `Largo = 3000`, → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 330: `Peso = 371.250m`, → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 331: `Activa = true`, → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 332: `}` → cierra el bloque de código actual.

Línea 333: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 334: `Detalle = new DetalleOrden` → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 335: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 336: `ComposicionQuimica = "C: 0.45%, Mn: 0.75%"`, → asigna la composición química utilizada para verificar la entidad DetalleOrden.

Línea 337: `TemperaturaColada = 1550.5`, → asigna la temperatura de colada del detalle de fabricación.

Línea 338: `Notas = "Colada principal"` → asigna las notas opcionales del detalle asociado a la orden.

Línea 339: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 340: `Certificado = new CertificadoCalidad` → crea el certificado uno-a-uno asociado a la orden del escenario.

Línea 341: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 342: `NumeroCertificado = "CERT-0001"`, → asigna el identificador natural del certificado de calidad.

Línea 343: `FechaEmision = DateTime.Today`, → asigna la fecha de emisión del certificado usado en la prueba.

Línea 344: `OrganismoCertificador = "Laboratorio Aceria"` → asigna el organismo que emite el certificado de calidad.

Línea 345: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 346: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 348: `var aleacion = new Aleacion` → crea la aleación que se enlazará con la orden mediante la entidad intermedia OrdenAleacion.

Línea 349: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 350: `Nombre = "AISI 1045"`, → asigna el nombre comercial de la aleación utilizada en el escenario.

Línea 351: `Codigo = "A1045"`, → asigna el código natural único de la aleación.

Línea 352: `PorcentajeCarbono = 0.45`, → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.

Línea 353: `PorcentajeManganeso = 0.75`, → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.

Línea 354: `Descripcion = "Acero medio en carbono"` → asigna una descripción opcional a la aleación.

Línea 355: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 356: `orden.OrganesAleaciones.Add(new OrdenAleacion` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 357: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 358: `Aleacion = aleacion`, → conecta la entidad intermedia `OrdenAleacion` con la aleación creada para el escenario.

Línea 359: `CantidadUtilizada = 1500.500m`, → registra la cantidad utilizada con tres decimales, coherente con `HasPrecision(18, 3)`.

Línea 360: `EstadoRelacion = "Activa"` → marca como activa la relación entre la orden y la aleación.

Línea 361: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 362: `context.OrganesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 363: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 364: `context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 366: `var entity = context.Model.FindEntityType(typeof(OrdenFabricacion));` → obtiene los metadatos EF Core de `OrdenFabricacion` para inspeccionar claves, índices o filtros del modelo construido.

Línea 367: `global::System.Console.WriteLine($"2.8 OK | Alternate keys: {entity.GetKeys().Count()} | PK: {string.Join(", ", entity.FindPrimaryKey().Properties.Select(x => x.Name))}");` → muestra la evidencia de ejecución de 2.8, el número de claves del modelo y las propiedades de la clave primaria.

Línea 368: `}` → cierra el bloque de código actual.

Línea 369: `}` → cierra el bloque de código actual.